Preguntas-Practicas-Primer-Parci...

Juandf03

Robótica

4º Grado en Ingeniería Informática

Escuela Técnica Superior de Ingeniería Informática
Universidad de Málaga

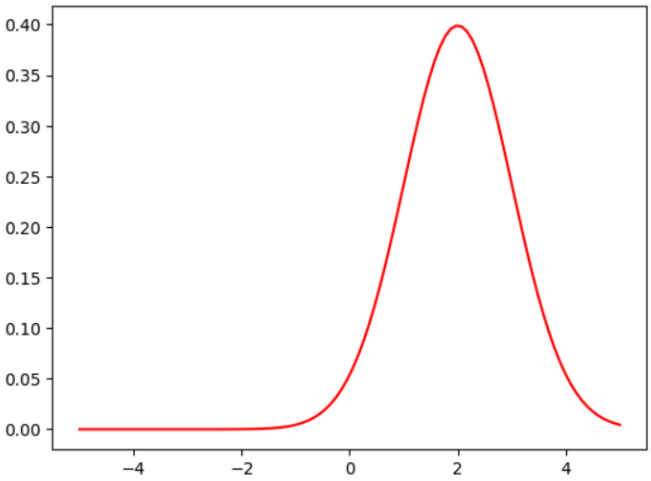
Accede al documento original



Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Tienes 7 días, ¿por qué conformarte con disfrutar solo dos? Entra en Randstad App y te ayudaremos a encontrar ese trabajo en el que te sientas bien.
descargar app
mola que estés así el sábado
pero mola más que estés así el lunes.
randstad
partner for talent.

1. Considérese la siguiente función que evalúa una función de densidad de probabilidad gaussiana (pdf). Tras la ejecución se obtiene el siguiente resultado:



Indica el valor de las variables empleadas en el código:
$\mu = 2$
min_interval = -5
max_interval = 5
n_samples = 100 (no puede ser 10)
plot(__, __, __) -> el tercer argumento es el color

```
# Create the array of values where the gaussian distribution is going to be evaluated
min_interval=-5
max_interval=5
n_samples=100
X = np.linspace(min_interval, max_interval, n_samples)

# Call the function and plot the results
res = evaluate_gaussian(mu, sigma, X)
plt.plot(X, res, 'r') # Show the results
plt.show() # Try what happens if you remove this line ;)
```

2. A lo largo de la asignatura, hemos utilizado frecuentemente la función *random.rand(n)* para generar muestras de una distribución de probabilidad gaussiana.
Cuando no tenemos una normal N(0,1) tenemos que sumar la media y multiplicar por la desviación típica.

WUOLAH
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```python
def gen_samples(n, mu, sigma):
    """Generate n samples of a gaussian distribution

    Args:
        n: Number of samples
        mu: mean of the distribution
        sigma: standard deviation of the distribution

    Returns:
        array of samples
    """

    samples = mu + sigma * np.random.randn(n)
    return samples
```

3. Esta función toma la pose x y la transforma de acuerdo con el movimiento u=[v,w]^T, aplicando el modelo de conducción diferencial.

El primer if comprueba que la matriz de covarianza no es nula. Las dimensiones de cov son 2x2.

random.randn(2,1) -> simplemente es el tamaño del vector que está generando, 2 filas, 1 columna.

Añade un ruido de media 0.

El segundo if comprueba que la velocidad angular es nula. Por su parte, R se corresponde con el radio que hay desde el centro del robot hasta el Centro Instantáneo de Rotación (ICR).

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Entre apuntes y exámenes, dale amor a tu coco con VivaGym.

```python
def next_pose(x, u, dt, cov=None):
    ''' This function takes pose x and transform it according to the motion u=[v,w]'
        applying the differential drive model.

        Args:
            x: current pose
            u: differential command as a vector [v, w]'
            dt: Time interval in which the movement occurs
            cov: covariance of our movement. If not None, then add gaussian noise
    '''
    if cov is not None:
        u += np.sqrt(cov) @ random.randn(2, 1)
        #u = np.random.multivariate_normal(u.flatten(),cov)

    if u[1] == 0: #linear motion w=0
        next_x = np.vstack([
            x[0] + u[0] * dt * np.cos(x[2]),
            x[1] + u[0] * dt * np.sin(x[2]),
            x[2]
        ])
    else: #Non-linear motion w=!0
        R = u[0]/u[1] #v/w=r is the curvature radius
        next_x = np.vstack([
            x[0] - R * np.sin(x[2]) + R * np.sin(x[2] + u[1] * dt),
            x[1] + R * np.cos(x[2]) - R * np.cos(x[2] + u[1] * dt),
            x[2] + u[1] * dt
        ])

    return next_x
```

4. $z = m - x$ **Mide distancias euclídeas de la pose del robot a los landmarks que hay en el mapa.**

No es una medición de distancia entre la posición del robot y un landmark, la m es el mapa del entorno completo formado por N marcas. z es un vector donde el elemento i-ésimo será la distancia entre el robot y el landmark m_i

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Entre apuntes y exámenes, dale amor a tu coco con VivaGym.

Tienes 7 días, ¿por qué conformarte con disfrutar solo dos? Entra en Randstad App y te ayudaremos a encontrar ese trabajo en el que te sientas bien.
descargar app
mola que estés así el sábado
pero mola más que estés así el lunes.

randstad
partner for talent.

```python
def distance(pose: np.ndarray, m: np.ndarray, cov_d: int = 0) -> np.ndarray:
    """ Get observations for every landmark in the map.

    In this case our observations are range only.
    If cov_d > 0 then add gaussian noise with var_d covariance

    Args:
        pose: pose (true or noisy) of the robot taking observation
        m: Map containing all landmarks
        cov_d: Covariance of the sensor

    Returns
        z: numpy array containing distances to all obs. It has shape (nLandmars, 1).
    """
    z = np.sqrt((m[:][0] - pose[0])**2 + (m[:][1] - pose[1])**2) # compute distances to all landmarks

    if cov_d > 0:
        # Calculate noise based on the distance to landmarks
        noise_std = np.sqrt(cov_d) * np.sqrt(z)  # Standard deviation of the noise
        noise = stats.norm(0, noise_std).rvs(size=z.shape)  # Generate Gaussian noise
        z += noise  # add noise if needed

    return z
```

5. Wz1 sensing incertidumbre asociada a la medida

Completar las demás preguntas

2x2 todo
3x3

Ordenar mínimos cuadrados (localización) -> bloques de código

EFKLocalization

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.